

Diccionarios

Un **diccionario** es una estructura de datos que permite almacenar información mediante un sistema de **pares clave-valor**. Es decir, cada elemento que contiene tiene una **clave (key)** única, asociada a un **valor (value)**. A este tipo de estructura también se lo conoce como **tabla de dispersión** o **mapa asociativo** en otros lenguajes de programación.

En muchos contextos reales, no basta con guardar datos en forma de lista o conjunto, ya que necesitamos asociar un dato con otro. Por ejemplo, si queremos guardar la edad de una persona, podríamos usar un diccionario donde la **clave** sea el nombre de la persona y el **valor** sea su edad. Así evitamos recorrer una lista buscando una posición y accedemos directamente al valor a través de su clave.

- Cada **clave** es **única** dentro del diccionario. No puede haber dos claves iguales.
- Los **valores** pueden ser de cualquier tipo de dato: números, cadenas, listas, otros diccionarios, etc.
- Los diccionarios son **mutables**: se pueden agregar, modificar o eliminar pares clave-valor después de su creación.
- Son **no ordenados** hasta versiones anteriores de Python 3.6. A partir de Python 3.7 en adelante, los diccionarios mantienen el orden de inserción, aunque no deben considerarse estructuras ordenadas en el mismo sentido que las listas.

Declaración de un diccionario

La forma más simple para crear un diccionario vacío, sin elementos iniciales, es:

```
diccionario_vacio = {}
```

O también usando la función **dict()**:

```
diccionario_vacio = dict()
```

Podemos crear un diccionario directamente con pares clave-valor, separando cada par por comas y usando dos puntos para asignar el valor a cada clave:

```
agenda = {  
    "Pedro": 11111111,  
    "María": 22222222,  
    "Clara": 33333333,  
    "Nicanor": 44444444  
}
```

No solo se pueden usar cadenas como claves, sino también otros tipos de datos **inmutables**, como números o tuplas.

Métodos de los diccionarios en Python

Los diccionarios en Python no solo almacenan pares clave-valor, sino que también cuentan con un conjunto de **métodos incorporados** que permiten manipular y consultar su contenido de forma eficiente y sencilla. Estos métodos están disponibles para cualquier diccionario y se utilizan para realizar operaciones comunes como agregar, eliminar, buscar, o modificar elementos, así como para obtener información sobre las claves o los valores que contiene.

Cuando hablamos de métodos, nos referimos a funciones que pertenecen a un objeto específico (en este caso, un diccionario) y que pueden ser llamadas para ejecutar una acción relacionada con ese objeto. Por ejemplo, un método puede añadir un nuevo par clave-valor, eliminar una clave existente, obtener una lista de todas las claves, o vaciar el diccionario por completo.

Estos métodos ya están definidos en la biblioteca estándar de Python, por lo que no es necesario programarlos desde cero. Solo basta con invocarlos usando la sintaxis **diccionario.método()** para que realicen la tarea deseada sobre el diccionario.

Python ofrece métodos muy prácticos para acceder a diferentes partes de un diccionario: las claves, los valores, o ambos al mismo tiempo. Estos métodos no modifican el diccionario, sino que permiten **recorrer o consultar** su contenido de forma estructurada.

Los tres métodos más comunes para este propósito son:

- **keys()**: devuelve un objeto iterable con todas las claves.
- **values()**: devuelve un objeto iterable con todos los valores.
- **items()**: devuelve un objeto iterable con todos los pares clave-valor como tuplas.

Estos métodos son especialmente útiles cuando se necesita **recorrer el diccionario en ciclos, buscar información o realizar transformaciones** sobre su contenido.

Método keys()

El método **keys()** se utiliza para **obtener todas las claves** que existen dentro de un diccionario. Devuelve un **objeto iterable** que se comporta como una vista dinámica del diccionario, lo cual significa que refleja los cambios que se hagan sobre las claves mientras el programa se ejecuta.

- No devuelve una lista, sino un objeto de tipo **dict_keys**. Sin embargo, si se necesita una lista, se puede convertir usando **list()**.
- Es útil para recorrer o consultar si cierta clave existe dentro del diccionario.

- Es ideal para usar en estructuras de control como **for**.

```
agenda = {  
    "Pedro": 11111111,  
    "María": 22222222,  
    "Clara": 33333333,  
    "Nicanor": 44444444  
}  
  
claves = agenda.keys()  
lista_claves = list(claves)  
  
print(lista_claves)
```

```
['Pedro', 'María', 'Clara', 'Nicanor']
```

Método values()

El método **values()** se utiliza para **obtener todos los valores** almacenados en un diccionario. Al igual que **keys()**, devuelve un **objeto iterable** especial llamado **dict_values**, que refleja en tiempo real los valores del diccionario.

- Devuelve todos los **valores asociados a las claves** del diccionario.
- No devuelve una lista, pero se puede convertir con **list()** si se desea.
- Puede contener valores repetidos, ya que los valores no tienen que ser únicos en un diccionario.
- Es útil para inspeccionar o recorrer los datos almacenados sin importar las claves.

```
agenda = {  
    "Pedro": 11111111,  
    "María": 22222222,  
    "Clara": 33333333,  
    "Nicanor": 44444444  
}  
  
valores = agenda.values()  
lista_valores = list(valores)  
  
print(lista_valores)
```

```
[11111111, 22222222, 33333333, 44444444]
```

Método items()

El método **items()** permite obtener **todos los pares clave-valor** del diccionario, agrupados en tuplas. Devuelve un objeto especial llamado **dict_items**, que es iterable y refleja el contenido actual del diccionario.

- Cada elemento del objeto **dict_items** es una **tupla** con dos elementos: la clave y su valor correspondiente.
- Es muy útil cuando se desea **recorrer un diccionario completo**, obteniendo tanto la clave como el valor al mismo tiempo.
- Se puede convertir a una lista de tuplas para trabajar más fácilmente si se requiere.

```
agenda = {  
    "Pedro": 11111111,  
    "María": 22222222,  
    "Clara": 33333333,  
    "Nicanor": 44444444  
}  
  
pares = agenda.items()  
lista_pares = list(pares)  
  
print(lista_pares)
```

```
[('Pedro', 11111111), ('María', 22222222), ('Clara', 33333333), ('Nicanor', 44444444)]
```